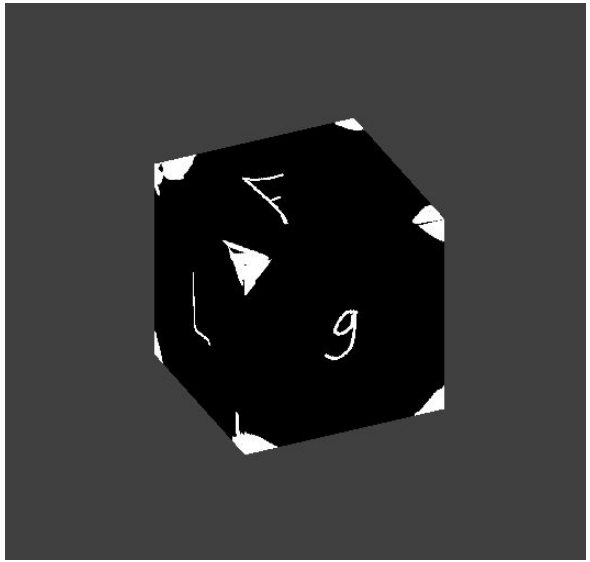
koka3D

Software OBJ rasterizer.



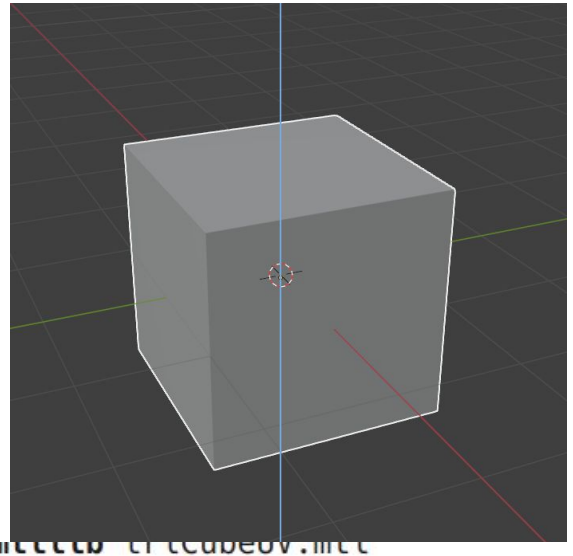
Part 1

OBJ :

- A text based 3D file format.
- Stores 3D vertices
- Stores Indices

PNG:

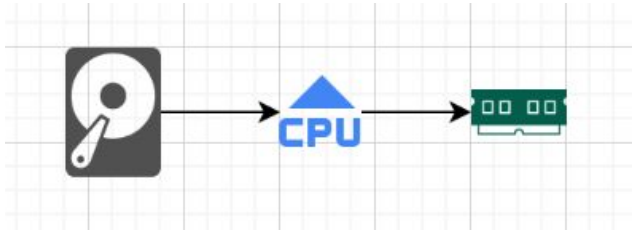
- Image file format

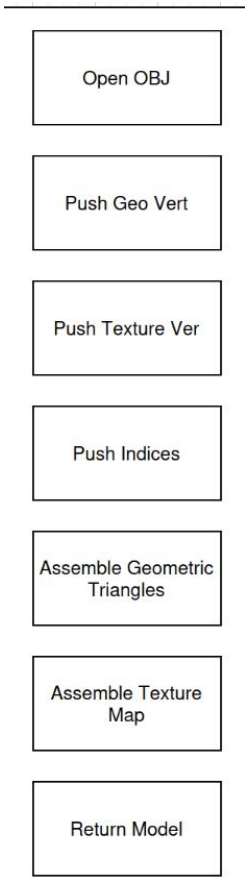


```
o Cube
v 1.000000 1.000000 -1.000000
v 1.000000 -1.000000 -1.000000
v 1.000000 1.000000 1.000000
v 1.000000 -1.000000 1.000000
v -1.000000 1.000000 -1.000000
v -1.000000 -1.000000 -1.000000
v -1.000000 1.000000 1.000000
v -1.000000 -1.000000 1.000000
```

Part 2

Pipeline: A series of processing where the output becomes the input for the next step.






```
o Cube
v 1.000000 1.000000 -1.000000
v 1.000000 -1.000000 -1.000000
v 1.000000 1.000000 1.000000
v 1.000000 -1.000000 1.000000
v -1.000000 1.000000 -1.000000
v -1.000000 -1.000000 -1.000000
v -1.000000 1.000000 1.000000
v -1.000000 -1.000000 1.000000
```

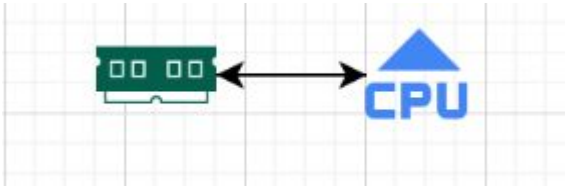
```
f 5/1/1 3/2/1 1/3/1
f 3/2/2 8/4/2 4/5/2
f 7/6/3 6/7/3 8/8/3
f 2/9/4 8/10/4 6/11/4
f 1/3/5 4/5/5 2/9/5
f 5/12/6 2/9/6 6/7/6
f 5/1/1 7/13/1 3/2/1
f 3/2/2 7/14/2 8/4/2
f 7/6/3 5/12/3 6/7/3
f 2/9/4 4/5/4 8/10/4
f 1/3/5 3/2/5 4/5/5
f 5/12/6 1/3/6 2/9/6
```

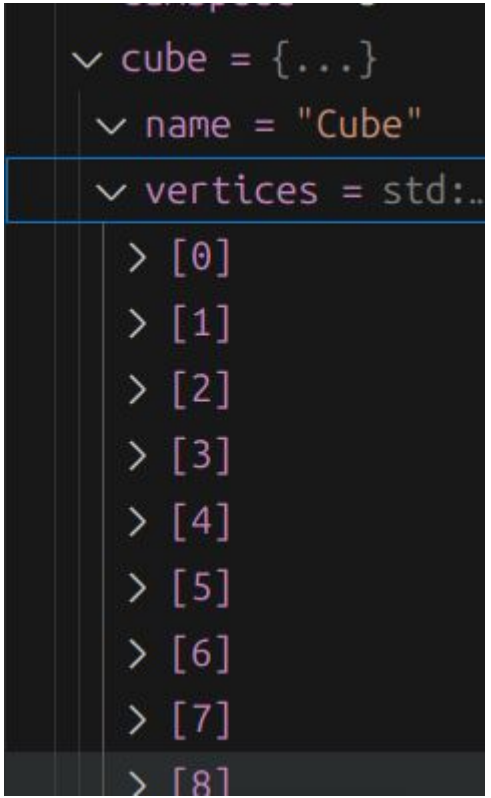
Part 3

SFML handles:

- Window creation.
- Window input.
- Telling the GPU to draw triangles.

I have a function that takes a model and window instance, then transforms it.

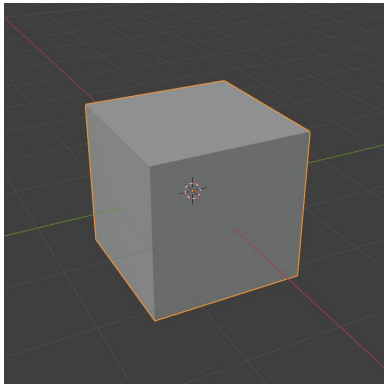


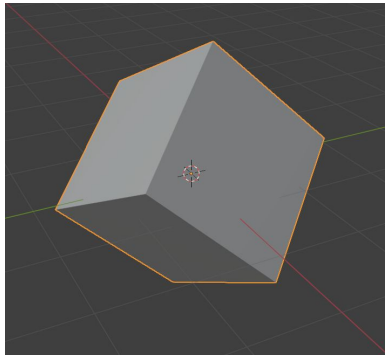


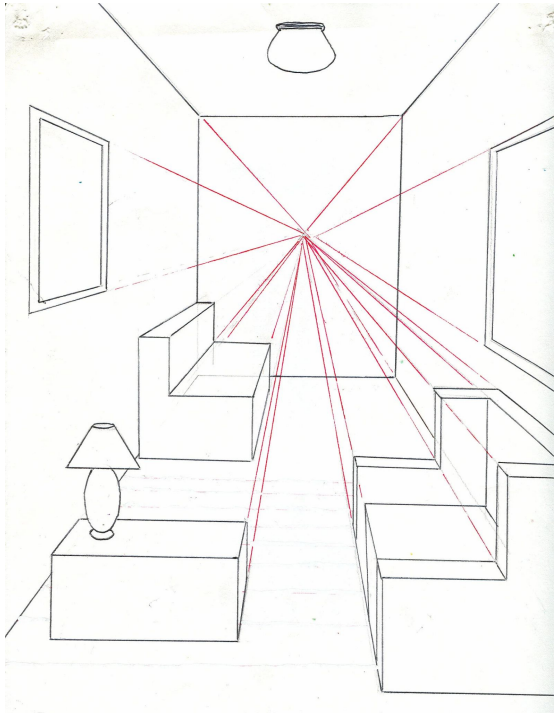
Part 4

Transformations:

- Translating
- Scaling
- Rotating
- Projecting
- Mapping textures.
- Sorting







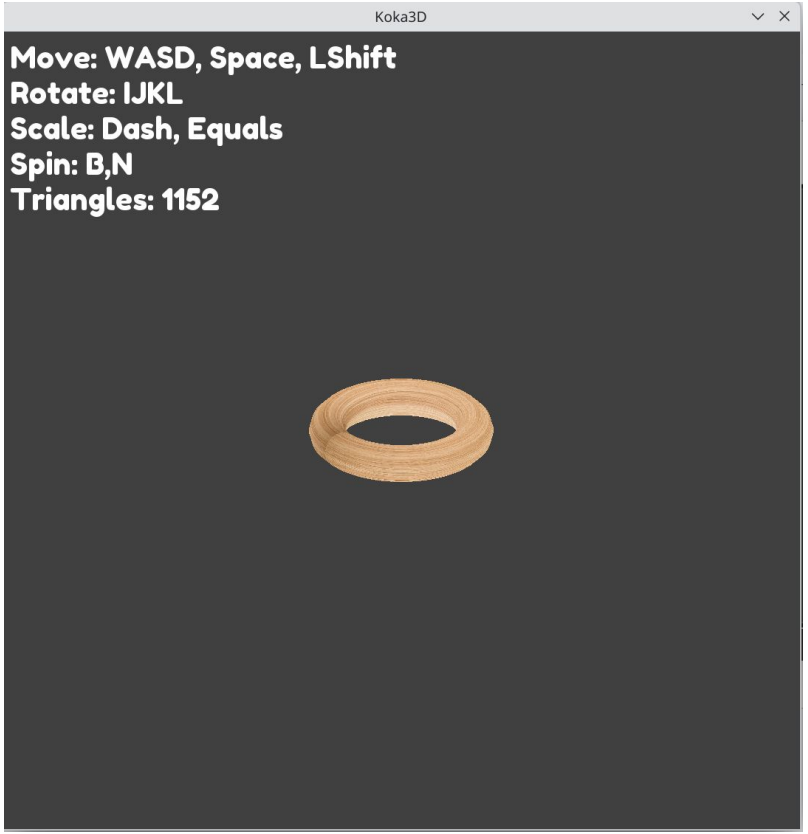
One Point Projection Accomplished with X/Z, Y/Z

Part 5

Rasterization:

Transformed vertices are send to a vertex queue where SFML tells GPU to draw these triangles. And to attach a texture to it.

Repeat previous step 60 times per second!

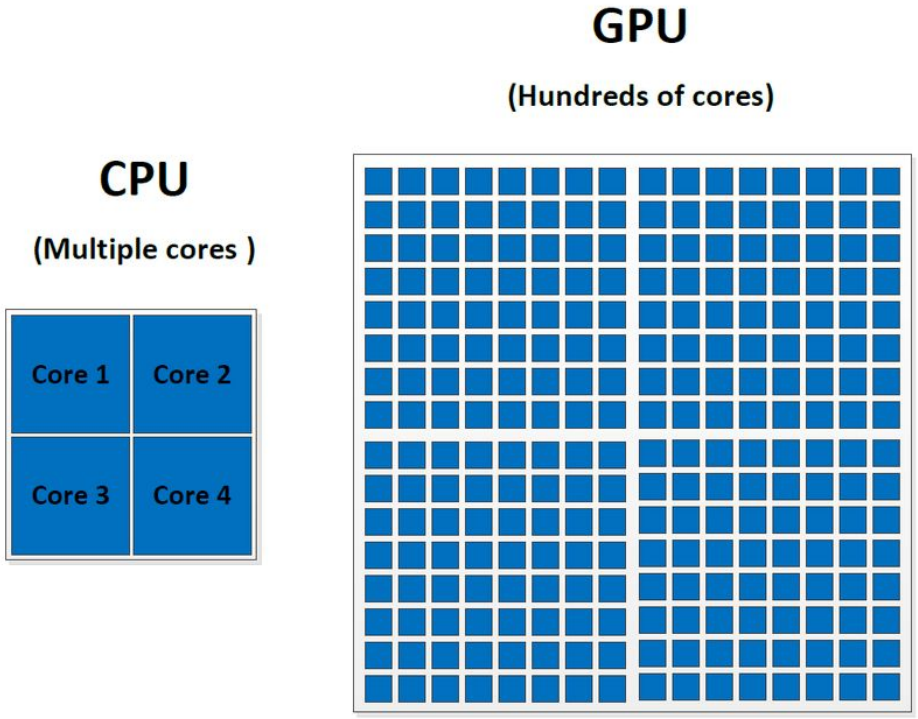


Software rendering sucks!

CPU can only transform a handful of vertices at a time. VERY SLOW!!!

GPU can transform hundreds of vertices all at once. VERY FAST!!!!

GPU API's handle many trivial tasks ie Sorting, Triangle culling automagically.



Why did I do this project?

Requires:

- Competency with file input/output.
- Precise data structures.
- Competent in linear algebra.
- Proficiency in low-level optimization.

Started June, v1.0.0 completed October,
~40hrs spent